

MIPS / PS2 — Decompilation Cheatsheet

Complete reference for the sly1 decomp project · TheOnlyZac/sly1 · Sly Cooper and the Thievius Raccoonus

1 · Integer registers (\$0 – \$31)

Number	Alias	Name	Role
\$0	\$zero	Zero	Always 0, immutable. <code>daddu \$s0,\$a0,\$zero</code> = copy \$a0 into \$s0
\$2–\$3	\$v0–\$v1	Value	Function return value (return x;). \$v0 = main integer result
\$4–\$7	\$a0–\$a3	Argument	Arguments 1–4. \$a0 = 1st arg, \$a1 = 2nd, etc.
\$8–\$11	\$a4–\$a7	Argument PS2	Arguments 5–8 — PS2 Emotion Engine extension, not present on x86/standard MIPS
\$12–\$15 \$24–\$25	\$t0–\$t9	Temporary	Clobbered by jal — never rely on these after a function call
\$16–\$23	\$s0–\$s7	Saved	Callee-saved — survive across jal calls. Used to preserve args before calls.
\$26–\$27	\$k0–\$k1	Kernel	Reserved for OS — never used in game code
\$28	\$gp	Global pointer	Points to global variables section (rarely seen in Sly)
\$29	\$sp	Stack pointer	Top of the stack. <code>addiu \$sp,\$sp,-N</code> allocates N bytes
\$30	\$fp/\$s8	Frame pointer	Rarely used as frame pointer in MIPS — often repurposed as a 9th saved register
\$31	\$ra	Return address	Return address. <code>jr \$ra</code> = return. Saved with <code>sq</code> if the function makes jal calls

2 · Floating-point registers (\$f0 – \$f31) — Coprocessor 1 (FPU)

Number	Alias	Name	Role
\$f0–\$f3	\$fv0–\$fv1	Float return	Floating-point return value (return 1.5f;)
\$f12–\$f15	\$fa0–\$fa1	Float args 1–4	First 4 float arguments — e.g. float t, float svt in <code>EvaluateAcph</code>
\$f20–\$f31	\$fs0–\$fs5	Saved float	Callee-saved — survive jal calls (like \$s0–\$s7 for integers)
\$f4–\$f11 \$f16–\$f19	\$ft0–\$ft7	Temp float	Caller-saved temporaries — clobbered after jal

Example `swc1 $f12, 4(a0)` Stores the float in \$f12 at address (a0+4) → `*(float*)(pObj+4) = val` | `lwc1` = reverse: loads a float from memory

3 · Floating-point arithmetic instructions (FPU)

Instruction	C equivalent	Note
<code>add.s \$fd,\$fs,\$ft</code>	<code>\$fd = \$fs + \$ft</code>	Float addition
<code>sub.s \$fd,\$fs,\$ft</code>	<code>\$fd = \$fs - \$ft</code>	Float subtraction
<code>mul.s \$fd,\$fs,\$ft</code>	<code>\$fd = \$fs * \$ft</code>	Float multiplication
<code>div.s \$fd,\$fs,\$ft</code>	<code>\$fd = \$fs / \$ft</code>	Float division (slow!)
<code>neg.s \$fd,\$fs</code>	<code>\$fd = -\$fs</code>	Float negation
<code>abs.s \$fd,\$fs</code>	<code>\$fd = fabsf(\$fs)</code>	Float absolute value
<code>sqr.s \$fd,\$fs</code>	<code>\$fd = sqrtf(\$fs)</code>	Square root
<code>mov.s \$fd,\$fs</code>	<code>\$fd = \$fs</code>	Copy float register
<code>cvt.s.w \$fd,\$fs</code>	<code>\$fd = (float)\$fs</code>	Convert int to float
<code>cvt.w.s \$fd,\$fs</code>	<code>\$fd = (int)\$fs</code>	Convert float to int (truncates)
<code>c.eq.s \$fs,\$ft</code>	<code>if (\$fs==\$ft) CC=1</code>	Float compare — result in Condition Code flag
<code>c.lt.s \$fs,\$ft</code>	<code>if (\$fs<\$ft) CC=1</code>	Float compare (less than)
<code>c.le.s \$fs,\$ft</code>	<code>if (\$fs<=\$ft) CC=1</code>	Float compare (less or equal)
<code>bc1t .label</code>	<code>if (CC==1) goto label</code>	Branch if float condition true
<code>bc1f .label</code>	<code>if (CC==0) goto label</code>	Branch if float condition false
<code>mfc1 \$rt,\$fs</code>	<code>\$rt (int) = \$fs (float)</code>	Move float to integer register (raw bits)
<code>mtc1 \$rt,\$fs</code>	<code>\$fs (float) = \$rt (int)</code>	Move integer to float register (raw bits)

4 · Memory instructions (load / store)

Instruction	Size	C type	Note
<i>lb / sb</i>	8 bits	signed char	Load/store 1 signed byte. <i>lbu</i> = unsigned variant
<i>lh / sh</i>	16 bits	short	Load/store 2 signed bytes. <i>lhu</i> = unsigned variant
<i>lw / sw</i>	32 bits	int / ptr	Most common — int, pointer, or float viewed as raw bits
<i>lwc1/swc1</i>	32 bits	float	Load/store to/from float register \$f
<i>ldc1/sdc1</i>	64 bits	double	Load/store double (pair of \$f registers)
<i>lq / sq</i>	128 bits	quadword PS2	EE-specific — used to save/restore \$s registers (128 bits each)
<i>ld / sd</i>	64 bits	doubleword	Load/store 64-bit integer

5 · Branches, jumps and integer arithmetic

Instruction	Effect / C equivalent
<i>jal FunctionX</i>	Call FunctionX, set \$ra = return address (+ delay slot)
<i>jalr \$2</i>	Call the function whose address is in \$2 — vtable / function pointer
<i>jr \$31 / \$ra</i>	return — jump to \$ra
<i>beqz \$r, .label</i>	if (\$r == 0) goto label → if (ptr == NULL) / if (x == 0)
<i>bnez \$r, .label</i>	if (\$r != 0) goto label
<i>beq \$r,\$s, .label</i>	if (\$r == \$s) goto label
<i>bne \$r,\$s, .label</i>	if (\$r != \$s) goto label
<i>bltz \$r, .label</i>	if (\$r < 0) goto label
<i>bgez \$r, .label</i>	if (\$r >= 0) goto label
<i>addiu \$d,\$s,N</i>	\$d = \$s + N (signed 16-bit immediate)
<i>daddu \$d,\$a,\$b</i>	\$d = \$a + \$b (64-bit) — equivalent of addu on the Emotion Engine
<i>subu \$d,\$a,\$b</i>	\$d = \$a - \$b
<i>and/or/xor/nor</i>	Bitwise operations — andi/ori/xori = immediate versions
<i>sll/srl/sra \$d,\$s,N</i>	Shift left / logical right / arithmetic right by N bits
<i>slt \$d,\$a,\$b</i>	\$d = (\$a < \$b) ? 1 : 0 — often followed by bnez to implement if
<i>mult/div \$a,\$b</i>	Result in HI:LO. Retrieve with mfhi/mflo

DELAY SLOT WARNING

The instruction AFTER a jump (*jal*, *jr*, *beqz*...) ALWAYS executes before the jump — this is PS2 hardware behavior. It is indented in the ASM listing. In C it is invisible (the compiler handles it). Practical rule: read the indented instruction FIRST, then the jump.

6 · C++ name mangling — reading mangled names

The compiler encodes the full signature into the function name. Format: `FunctionName__[qualifiers]Args`

Mangled suffix	Decoded type	Explanation
<i>Fv</i>	void	No arguments (<i>Fv</i> = Function with void args)
<i>Fi</i>	int	One int argument
<i>Ff</i>	float	One float argument
<i>Fc</i>	char	One char argument
<i>FPc</i>	char*	Pointer to char (P = Pointer)
<i>FP10CLOCK</i>	CLOCK*	Pointer to struct — the number is the length of the type name
<i>FP3AOffi</i>	ALO*, float, float, int	Multiple args — read left to right
<i>T3</i>	(repeat of 3rd arg)	T + number = repeat the type of argument N — same type as arg N
<i>R3</i>	(ref to 3rd type)	R + number = reference to a previously seen type
<i>__F...</i>	free function	No this — global or static function
<i>__CF...</i>	const method	Const method (this = const pointer)

Mangled name (in .s files)	Decoded C++ signature
<i>ResetClock__FP5CLOCKf</i>	<code>void ResetClock(CLOCK*, float)</code>

Mangled name (in .s files)	Decoded C++ signature
GetDifficulty__Fv	int/enum GetDifficulty(void)
EvaluateAcpb__FP4ACBPBP3ALOffiP6VECTORT5	void EvaluateAcpb(ACPB*, ALO*, float, float, int, VECTOR*, VECTOR*)
OnDifficultyGameLoad__FP10DIFFICULTY	void OnDifficultyGameLoad(DIFFICULTY*)

Tip c++filt MangledName in the terminal decodes automatically. Or paste on <https://demangler.com>

7 · Reading a struct from ASM — finding field offsets

Every lw/sw reveals a struct field. The offset in the instruction = byte position of that field in the struct.

ASM instruction	Field (guess)	What we can infer
lw \$2, 0x00(\$4)	field_0x00	Offset 0 — often the vtable pointer (C++) or first member
lw \$2, 0x04(\$4)	field_0x04	Second field (if the first was 4 bytes)
lw \$2, 0x0C(\$4)	field_0x0C	Offset 12 — e.g. ACPB::ckvb in EvaluateAcpb
lw \$2, 0x10(\$4)	field_0x10	Offset 16 — e.g. ACPB::akvb (KVB* pointer)
lw \$2, 0xB0(\$4)	vtable[0xB0]	Vtable entry — virtual method #44 (0xB0/4=44)
lwc1 \$f0, 0x08(\$4)	field_0x08	Float field at offset 8 — lwc1 means it's a float, not an int
lbu \$2, 0x01(\$4)	field_0x01	Unsigned char/bool at offset 1

#	Step	Detail
1	List all lw/sw	Each unique offset = a different struct field
2	Sort by offset	Field order in the struct = ascending offsets
3	Infer the type	lw=int/ptr, lwc1=float, lbu=char, lhu=short, lq=128-bit vector
4	Check inheritance	ACPB : public ACP → ACP's fields come first (offset 0)
5	Name the fields	Cross-reference with Ghidra + other functions accessing the same struct

8 · Common patterns in Sly 1

Pattern	Typical ASM	C equivalent
Prologue	addiu \$sp,\$sp,-N sq \$ra, X(\$sp) sq \$sN, Y(\$sp)	Allocates N bytes on the stack. Saves \$ra if the function makes calls. Saves the \$s registers it will use.
Epilogue	lq \$sN, Y(\$sp) lq \$ra, X(\$sp) jr \$ra addiu \$sp,\$sp,N	Restores \$s and \$ra. Frees the stack. The addiu is in the delay slot of jr.
Arg copy	daddu \$sN, \$aM, \$zero	Copies an arg (\$a) into a \$s before a jal that would clobber \$a. Very common.
Vtable call	lw \$2, 0x0(\$obj) lw \$2, 0xXX(\$2) jalr \$2	(*obj->vtable[offset])() — C++ virtual method. 1st lw = deref vtable, 2nd = fetch fn ptr, jalr = call.
Null check	beqz \$reg, .end	if (ptr == NULL) return — classic guard before accessing a pointer or calling a vtable.
Field access	lw \$2, 0x10(\$4)	\$2 = pObj->field_0x10 — the offset is the byte displacement into the struct.
For loop	addiu \$t0,\$zero,0 (init) bne \$t0,\$t1,.loop (test) addiu \$t0,\$t0,1 (increment)	Typical for(int i=0; i<n; i++) — the three parts are often spread around the loop body.
Switch/table	sll \$2,\$2,2 add \$2,\$2,\$base lw \$2,0(\$2) jr \$2	Switch via jump table. Multiplies index by 4, indexes an array of pointers, jumps.

9 · Annotated example — EvaluateAcpb (ASM to C walkthrough)

A real function from the project — 30 lines of ASM translated to ~10 lines of C.

ASM	Pattern	Translation
addiu \$sp,\$sp,-0x40	prologue	Allocates 64 bytes on the stack
sq \$18,0x20(\$sp) sq \$17,0x10(\$sp) sq \$16,0x0(\$sp) sq \$31,0x30(\$sp)	save regs	Saves \$s2,\$s1,\$s0,\$ra — this function makes a jal
daddu \$18,\$8,\$0 daddu \$17,\$7,\$0 daddu \$16,\$5,\$0	copy args	\$s2=grfeval \$s1=svt \$s0=palo — will survive the upcoming jal
lw \$5,0x10(\$4) jal EvaluateAkvb lw \$4,0xC(\$4)	call 1	EvaluateAkvb(pacpb->ckvb, pacpb->akvb, t, svt, grfeval, ppos, pv) Delay slot: the lw executes BEFORE the jal!
beqz \$16,.end lq \$31,0x30(\$sp)	null check	if (palo==NULL) goto end — delay slot restores \$ra during the test

ASM	Pattern	Translation
<i>lw \$2,0x0(\$16) lw \$2,0xB0(\$2)</i>	vtable	\$2 = *(palo->vtable + 0xB0) — fetch virtual method pointer
<i>beqz \$2,.end</i>	null check 2	if (func==NULL) goto end — vtable slot may be empty
<i>daddu \$4,\$16,\$0 daddu \$5,\$17,\$0 jalr \$2 daddu \$6,\$18,\$0</i>	vtable call	func(palo, ppos, pv) via pointer Delay slot: sets up \$a2=pv during the jump
<i>lq \$18,... lq \$17,... lq \$16,... jr \$31 addiu \$sp,\$29,0x40</i>	epilogue	Restores \$s2,\$s1,\$s0. return. Delay slot frees the stack.

Resulting C code	Explanation
<i>void EvaluateAcpb(ACPB *pacpb, ALO *palo, float t, float svt, GRFEVAL grfeval, VECTOR *ppos, VECTOR *pv)</i>	Signature — taken directly from ac.h. 7 args = \$a0–\$a3 + \$a4(grfeval) + 2 VECTOR* in \$a5/\$a6
<i>EvaluateAkvb(pacpb->ckvb, pacpb->akvb, t, svt, grfeval, ppos, pv);</i>	Direct call. ckvb at offset 0x0C, akvb at offset 0x10 in ACPB (after ACP parent fields)
<i>if (palo != NULL)</i>	Null check on \$s0 (= palo saved before the jal)
<i>void (*func)(...) = *(void(**)(...))((char*)((void**)palo)) + 0xB0);</i>	Deref vtable + offset 0xB0 = virtual method #44
<i>if (func != NULL) func(palo, ppos, pv);</i>	Conditional call — some ALOs don't implement this method (null vtable slot)

10 · Ghidra — practical tips for Sly 1

Shortcut / Action	Name	Use for decomp
<i>G</i>	Go to address	Jump to a hex address (e.g. 0011FDD8) — go straight to a function
<i>F</i>	Search functions	Search function by name — useful after importing symbols
<i>L</i>	Rename symbol	Rename a variable or function — essential for documenting as you reverse
<i>T</i>	Retype variable	Change a variable's type — key to reading structs correctly
<i>Ctrl+Shift+E</i>	Edit function sig	Edit the full function signature (name, args, return type)
<i>Ctrl+F</i>	Find in listing	Search in the ASM listing
<i>Ctrl+Shift+F</i>	Search all	Global search across the whole program (slow but powerful)
<i>X (xref)</i>	Find references	Who calls this function? Who accesses this variable? Indispensable.
<i>Ctrl+Z</i>	Undo	Undo renames/retypes — experiment freely
<i>Decompiler</i>	Pseudo-C window	Shows approximate C next to ASM — starting point for matching
<i>Import .yaml</i>	Load symbols	Ghidra script to import symbol_addrs.txt — official names in the UI

Ghidra workflow	1) Open SCUS_971.98 2) Import project symbols (scripts/) 3) Find function by address (G) 4) Read pseudo-C 5) Compare with .s file 6) Rename variables (L+T) to clarify 7) Copy pseudo-C into diff.sh and iterate
------------------------	--

11 · Common errors and troubleshooting

Error	Symptom	Fix
checksum failed	Compiled binary does not match the original	The C logic is wrong — wrong operation order, missing cast, wrong type. Re-check the diff.
undefined reference to 'X__FP...'	Linker cannot find the function	Missing or wrong entry in symbol_addrs.txt. Verify the exact mangled name and address.
ninja: no work to do (diff.sh)	Script cannot find the function	Wrong mangled name in the command. Check with grep in symbol_addrs.txt.
Matches decomp.me but not locally	Local compiler differs	GCC version or flags mismatch. Open a GitHub issue or ask on Discord.
Registers shifted in diff	Function compiles but registers differ	Declaration order or cast issue. Try reordering local variables. Try volatile.
Delay slot mismatch	Instruction in delay slot doesn't match	Reorder calls or assignments so the compiler optimizes in the right order.
lq/sq instead of lw/sw	Ghidra shows unexpected 128-bit accesses	Normal on PS2 EE — \$s registers are 128 bits. In C they remain normal 32/64-bit types.
Wrong struct offset	Fields are not at the expected offsets	Check inheritance (parent fields come first) and alignment (float/ptr aligned to 4 or 8 bytes).

12 · Full workflow — from zero to pull request

#	Step	Command / Action
1	Find a function	<code>wc -l asm/nonmatchings/**/*.*s sort -n</code> → pick one under 40 lines, few jal, readable name
2	Read the ASM	Identify: prologue, saved args, calls (jal), null checks, vtable, epilogue
3	Research	grep the struct in include/ for types. Use Ghidra for a pseudo-C starting point.
4	Wrap	In the .c file: <code>#ifdef SKIP_ASM / function / #endif</code> below <code>INCLUDE_ASM</code>
5	Diff	<code>./scripts/diff.sh MangledName File</code> → live diff. Iterate until 100% match.
6	Integrate	Remove <code>INCLUDE_ASM + #ifdef</code> . Add to <code>symbol_addrs.txt</code> if missing.
7	Validate	Full build — sha1sum must pass. If not: see Troubleshooting section.
8	Pull request	Fork → branch → commit → pull request on GitHub. Describe the function and its logic.

13 · PS2 EE calling convention — how arguments are passed

The calling convention defines who puts what where before and after a function call. Essential for reading ASM correctly.

Category	Registers used	Detail
Integer / pointers	<i>\$a0–\$a3 (\$4–\$7) then \$a4–\$a7 (\$8–\$11)</i>	PS2 passes up to 8 integer args in registers — no stack spilling for the first 8
Floats	<i>\$f12,\$f13,\$f14,\$f15 (then \$f16,\$f17...)</i>	Float args use FPU registers — independent of \$a slots. A float does not consume an \$a register.
Mixed int+float	<i>EvaluateAcpb(ACPB*,ALO*,float,float,int,VECTOR*,VECTOR*)</i>	<i>\$a0=pacpb \$a1=palo \$f12=t \$f13=svt \$a2=grfeval \$a3=ppos \$a4=pv</i> — int and float use separate slots
Return value	<i>Integer: \$v0 / Float: \$f0</i>	Void functions: nothing. Two return values: <i>\$v0+\$v1</i> or <i>\$f0+\$f2</i> .
More than 8 ints	<i>Passed on the stack (\$sp + offset)</i>	Rare in Sly — most functions have fewer than 8 args. lw at a large \$sp offset = stack argument.
Structs by value	<i>Copied on stack or passed by pointer</i>	In practice in Sly: always by pointer (VECTOR*, MATRIX3*). No direct struct copy in arguments.
Caller-saved	<i>\$a0–\$a7, \$t0–\$t9, \$v0–\$v1, \$f4–\$f19</i>	Caller must save these if needed after jal — hence <i>daddu \$s0,\$a0,\$zero</i> before calls
Callee-saved	<i>\$s0–\$s7, \$f20–\$f31, \$ra, \$sp</i>	Called function MUST restore these before returning — hence <i>sq/lq</i> in prologue/epilogue

C prototype	Register mapping	Note
<code>void Foo(int a, float b, int c, VECTOR *v)</code>	<i>\$a0=a \$f12=b \$a1=c \$a2=v</i>	int and float have separate slots — c is \$a1 not \$a2!
<code>int Bar(float x, float y, float z)</code>	<i>\$f12=x \$f13=y \$f14=z return->\$f0</i>	3 floats → 3 consecutive FPU registers
<code>void Baz(ALO*,float,float,int,VECTOR*,VECTOR*)</code>	<i>\$a0 \$f12 \$f13 \$a1 \$a2 \$a3</i>	Exact pattern of <i>EvaluateAcpb/Acpb</i> — very common in <i>ac.c</i>

14 · Special registers HI / LO — multiply, divide and accumulator

HI and LO are two special 64-bit registers that receive the results of mult and div. They cannot be used directly — you must use mfhi/mflo to read them.

Instruction	Effect	Note
<code>mult \$a, \$b</code>	<i>HI:LO = \$a * \$b (signed 64-bit)</i>	64-bit result: LO = low 32 bits, HI = high 32 bits
<code>multu \$a, \$b</code>	<i>HI:LO = \$a * \$b (unsigned)</i>	Unsigned variant of mult
<code>div \$a, \$b</code>	<i>LO = \$a / \$b HI = \$a % \$b</i>	Quotient in LO, remainder in HI — both in a single op
<code>divu \$a, \$b</code>	<i>LO = \$a / \$b HI = \$a % \$b (u)</i>	Unsigned variant
<code>mflo \$dst</code>	<i>\$dst = LO</i>	Move From LO — retrieve mult result or quotient
<code>mfhi \$dst</code>	<i>\$dst = HI</i>	Move From HI — retrieve high bits of mult or remainder
<code>mtlo \$src</code>	<i>LO = \$src</i>	Move To LO — rare, mainly used with macc
<code>mtli \$src</code>	<i>HI = \$src</i>	Move To HI
<code>madd \$a, \$b</code>	<i>HI:LO += \$a * \$b</i>	Multiply-Accumulate — accumulates multiple products
<code>msub \$a, \$b</code>	<i>HI:LO -= \$a * \$b</i>	Multiply-Subtract accumulator

Instruction	Effect	Note
<i>mul \$d,\$a,\$b</i>	<i>\$d = (\$a * \$b) & 0xFFFFFFFF</i>	Pseudo-instruction: mult + mflo in one — destroys HI:LO

Typical mult/div pattern

mult \$a0, \$a1 → mflo \$v0 → return \$v0 = return a * b | div \$a0,\$a1 → mflo \$v0 → mfhi \$v1 = quotient in \$v0, remainder in \$v1

15 · VU0 / VU1 and PS2 SIMD instructions (128-bit vectors)

The PS2 has two Vector Units (VU0, VU1) for vector math. VU0 is accessible inline from the main CPU via COP2 instructions.

Concept / Instruction	Syntax / Registers	Role
VF registers	<i>\$vf0–\$vf31</i>	32 x 128-bit vector registers — each = 4 floats (x, y, z, w)
VI registers	<i>\$vi0–\$vi15</i>	16 x 16-bit integer registers — indices, counters inside VU programs
VCLIP	<i>COP2 instruction</i>	3D clipping test — indicates which components exceed the view frustum
VADD	<i>vfd = vfs + vft</i>	Vector add — all 4 components x,y,z,w in parallel
VSUB	<i>vfd = vfs - vft</i>	Vector subtract
VMUL	<i>vfd = vfs * vft</i>	Component-wise multiply (NOT dot product!)
VMADD	<i>vfd = vfs*vft + ACC</i>	Multiply-add with vector accumulator
VOPMULA	<i>ACC = vfs x vft</i>	Partial cross product — stores in accumulator
VOPMSUB	<i>vfd = vfs x vft - ACC</i>	Full cross product with subtract — computes the real 3D cross product
VITOF0/FTOI0	<i>int(float)</i>	Vectorized int/float conversion across all 4 components
LQC2/SQC2	<i>128-bit load/store</i>	Load/store a VF register from/to memory — like lq/sq but targets VF regs
CTC2/CFC2	<i>CPU-VU0 transfer</i>	Move between CPU integer registers and VU0 control registers

In Sly 1

Most game logic uses normal scalar floats (FPU). VU instructions appear mainly in the renderer and matrix math. For decomp: lq/sq on \$s registers = normal callee-save (not VU). LQC2/SQC2 = actual VU vector access.

16 · Type and module glossary — complete overview of the P2 engine

Overview of the P2 engine architecture. Each .c file corresponds to one subsystem. Knowing their role helps you target the right functions.

Math, geometry and low-level types

Type	Source file	Role and key functions
<i>VECTOR</i>	<i>vec.h / P2/vec.c</i>	4 floats (x,y,z,w). 16 bytes. Position, direction, color. Passed by pointer VECTOR*. Key fns: SetVectorCylind, SetVectorSphere, SProjectVector, GetNormalVector
<i>MATRIX3</i>	<i>mat.h / P2/mat.c</i>	3x3 rotation matrix (9 floats, 36 bytes). Object orientation. MatMulMatTransMat, LoadIdentityMatrix3
<i>MATRIX4</i>	<i>mat.h / P2/mat.c</i>	4x4 full transform matrix (16 floats). Position + rotation. LoadIdentityMatrix4, multiply operator
<i>VU_FLOAT</i>	<i>splice/vecmat</i>	Float wrapper for VU ops — used in Splice scripting. Implicitly cast to float.
<i>VU_VECTOR</i>	<i>splice/vecmat</i>	VECTOR wrapper for VU — used in Splice to pass vectors to VU functions.
<i>LSG</i>	<i>P2/clip.c</i>	Line Segment — segment between two points. Result of clipping functions. ClsgClipEdgeToObject
<i>BSP</i>	<i>P2/bsp.c</i>	Binary Space Partition — space-partitioning tree for geometry. PruneBsp, PbspPointInBspQuick
<i>CBSP/CGT</i>	<i>P2/path.c</i>	Convex BSP / Convex Graph Tree — structures for navigation and pathfinding in the level.
<i>GEOM</i>	<i>P2/geom.c</i>	Object geometry (collision mesh). InitGeom, CloneGeom. Contains BSP and BSPC for intersection tests.
<i>CONSTR</i>	<i>P2/crv.c</i>	Constraint — path constraint curve. FindAposClosestPointAll, EvaluateAposG. Used for rails and paths.

World object system (LO / ALO / SO / SW)

All game objects fit into a hierarchy. SW is the world, LO is the base container, ALO is the animated object, SO adds physics.

Type	Source file	Role and key functions
<i>SW</i>	<i>P2/sw.c</i>	World — the current game world. Root container for all objects. InitSw, DeleteSw, SetSwGravity. Each level = one SW.
<i>LO</i>	<i>P2/lo.c</i>	Live Object — base game object. InitLo, AddLo, RemoveLo, PostLoLoad. All objects inherit from LO.
<i>ALO</i>	<i>P2/alo.c</i>	Animated Live Object — LO with animation. Has vtable, VECTOR position, MATRIX3, ASEG list. InitAlo, AddAloHierarchy. Base of Sly, guards, enemies.
<i>SO</i>	<i>P2/so.c</i>	Solid Object — ALO with physics and collision. InitSo, EnableSoPhys, DisplaceSo, ImpactSo. Objects that bounce/collide.
<i>OID</i>	<i>common.h</i>	Object ID — unique identifier for a world object (32-bit int). Used to reference objects in Splice scripts.

Type	Source file	Role and key functions
<i>CID</i>	<i>common.h</i>	Class ID — object class identifier (type). <i>FlsBasicDerivedFrom</i> tests if an object belongs to a given class.
<i>BASIC</i>	<i>P2/basic.c</i>	Base class for all scriptable objects. <i>FlsBasicDerivedFrom</i> , <i>EnsureBasicSidebag</i> , <i>GetBasicCid</i> . Bridge between C++ engine and Splice.
<i>MSGID / MSG</i>	<i>common.h</i>	Message ID — inter-object messaging system. <i>HandleXxxMessage</i> receives a <i>MSGID</i> + <i>void*</i> payload. Event-driven engine pattern.

Animation system (AC* / ASEG / ACT / BLEND)

Type	Source file	Role and key functions
<i>ASEG</i>	<i>P2/aseg.c</i>	Animation Segment — animation segment for an object. <i>InitAseg</i> , <i>LoadAsegFromBrx</i> , <i>CloneAseg</i> . Contains AC* curves for position/rotation/scale.
<i>ASEGA</i>	<i>P2/asega.c</i>	Animation Segment Active — ASEG being played back. <i>SetAsegaHandsOff</i> , <i>UpdateAsegaleaCur</i> . Manages real-time playback of an ASEG.
<i>ACT</i>	<i>P2/act.c</i>	Action — animation sequence for an object. <i>PactNew</i> , <i>InitAct</i> , <i>RetractAct</i> , <i>GetActPositionGoal</i> . Orchestrates the ASEGs of an ALO.
<i>ACTSEG</i>	<i>P2/actseg.c</i>	Action Segment — part of an ACTION. <i>GetActsegPositionGoal</i> , <i>GetActsegRotationGoal</i> , <i>GetActsegScale</i> . Provides animation goals.
<i>ACP</i>	<i>P2/ac.c</i>	Animation Curve Position — base class for position curves. <i>EvaluateAcp</i> , <i>GetAcpTimes</i> . Subclasses: <i>ACPB</i> (Bezier), <i>ACPC</i> (Component), <i>ACPBL</i> (Blend).
<i>ACR</i>	<i>P2/ac.c</i>	Animation Curve Rotation — rotation curves. <i>EvaluateAcr</i> , <i>EvaluateAcrEul</i> . Subclasses: <i>ACRB</i> , <i>ACRC</i> , <i>ACRBL</i> .
<i>ACS</i>	<i>P2/ac.c</i>	Animation Curve Scale — scale curves. <i>EvaluateAcs</i> , <i>GetAcsTimes</i> . Subclasses: <i>ACSB</i> , <i>ACSC</i> .
<i>ACG</i>	<i>P2/ac.c</i>	Animation Curve Generic — generic float curve. <i>EvaluateAcg</i> . Subclasses: <i>ACGB</i> , <i>ACGL</i> , <i>ACGBW</i> , <i>ACGBLT</i> , <i>ACGBLP</i> .
<i>KVB</i>	<i>P2/ac.c</i>	Key Value Bezier — scalar Bezier keyframe. Atomic building block of curves. <i>EvaluateAkvb</i> , <i>GetAkvbTimes</i> . Contains time, value, tangents.
<i>GRFEVAL</i>	<i>P2/ac.c</i>	Evaluation flags (int bitfield). Controls interpolation, extrapolation, looping. Passed to all <i>Evaluate*</i> functions. 0 = standard evaluation.
<i>BLEND</i>	<i>P2/blend.c</i>	Animation blending. <i>EnsureAsegBlendDynamic</i> , <i>CalculateBlendAmrsg</i> . Blends multiple ASEGs for transitions and procedural animation.
<i>MMSG</i>	<i>P2/blend.c</i>	Morph Segment — morphing segment. <i>ImrsgLookUp</i> . Used to blend between poses (e.g. idle to run).
<i>RCHM</i>	<i>P2/rchm.c</i>	Reach Map — reach map for IK. <i>BuildRchmCoefficients</i> , <i>ReblendRchm</i> . Computes IK weights for Sly's hands/feet.
<i>IK/IKH</i>	<i>P2/ik.c</i>	Inverse Kinematics. <i>SolveAloIK</i> , <i>RenderIkhSelf</i> . Solves limb poses so hands/feet touch surfaces correctly.

Physics, gameplay and movement (JT / XP / DZG)

Type	Source file	Role and key functions
<i>JT</i>	<i>P2/jt.c</i>	Jumping Thing — Sly Cooper's physical entity (the player). <i>InitJt</i> , <i>PresetJtAccel</i> , <i>AdjustJtNewXp</i> . Core of Sly's movement.
<i>XP</i>	<i>P2/mark.c</i>	Contact point — contact point between Sly and the world. <i>PxpAdd</i> , <i>RecalcSwXpPos</i> , <i>FreeSwXpList</i> . Each touched surface = one XP.
<i>DZ / DZG</i>	<i>P2/dzg.c</i>	Dead Zone / DZ Group — physical constraint zone. <i>SolveDzg</i> , <i>AppendDzgDz</i> . Solves constraints between XP contacts and surfaces.
<i>FX</i>	<i>P2/solve.c</i>	Force eXchange — force exchange between objects. <i>CalculateSoEffect</i> , <i>PropagateSoForce</i> . Collision and reaction physics.
<i>XA</i>	<i>P2/water.c</i>	eXternal Acceleration — external acceleration (wind, water, etc). <i>AddWaterExternalAccelerations</i> , <i>UpdateSwXaList</i> .
<i>PO</i>	<i>P2/po.c</i>	Player Object — object linked to the player. <i>InitPo</i> , <i>HandlePoMessage</i> , <i>GetPoCpdefi</i> . Manages the player-world relationship.
<i>CM</i>	<i>P2/cm.c</i>	Camera — game camera. <i>CalcCmAdjust</i> , <i>SpringCm</i> , <i>GetCmCpdefi</i> . Manages camera position and tracking.
<i>CPDEFI</i>	<i>P2/cm.c</i>	Camera Position DEFInition — camera position definition. <i>AdjustCmCpdefi</i> , <i>GetCmCpdefi</i> . Camera parameters for each situation.
<i>STEP</i>	<i>P2/step.c</i>	Step — Sly's locomotion system. <i>InitStep</i> , <i>PostStepLoad</i> , <i>LimitStepHands</i> . Orchestrates sub-systems (stepcane, steprun, stephide...).
<i>JOY</i>	<i>P2/joy.c</i>	Joystick — controller input. <i>StartupJoy</i> , <i>UpdateJoy</i> , <i>AddGrfusr</i> . Translates DualShock input into movement vectors.

Type	Source file	Role and key functions
ROV	P2/rog.c	Rover — wheeled vehicle (RC tank). InitRov, UpdateRovActive, ReadRovJoystick. Controls the RC tank in Police HQ.

Rendering (GLOB / GIFS / VIFS / SHD / RPL)

Type	Source file	Role and key functions
GLOB	P2/glob.c	Globe — 3D mesh of an object. LoadGlobsetFromBrx, BuildSubglobSinglePass. Contains vertices, UVs, normals.
GLOBSET	P2/glob.c	Set of GLOBs for an object. BuildSubcel, BuildSubglobSinglePass. Every visible object has a GLOBSET.
SHD	P2/shd.c	Shader / texture. Tex0FromTexIframeCtk, PackTexGifs, LoadBmpFromBrx. Contains texture + GS render parameters.
BMP	P2/shd.c	Bitmap — raw image loaded from BRX. LoadBmpFromBrx, LoadBitmapsFromBrx.
CLUT	P2/shd.c	Color Look-Up Table — palette for indexed PS2 textures. LoadClutFromBrx.
GIFS	P2/gifs.c	GIF Packet Stream — GIF packets sent to the GS (Graphics Synthesizer). AddPrimBulk, EndPrim. Low-level PS2 graphics interface.
VIFS	P2/vifs.c	VIF Packet Stream — VIF packets sent to the VUs. AddVifMscal, AddVifBaseOffset. Uploads mesh data to VU1.
GLBS	P2/glbs.c	Global Strip — global render strip. SetTransform, FindLights, BeginStrip, DrawPrelit. Renders a strip of polygons.
RPL	P2/render.c	Render PLane — sorted render plane. SubmitRpl, NCmpPrplReverseZ, CprplPartitionArpl. Sorts objects by Z for rendering.
SAA	P2/shdanim.c	Shader Animation Array — texture animation (UV scroll, palette anim). PsaaLoadFromBrx, InitSaa, FUdatableSaa.
DYSH	P2/dysh.c	Dynamic SHader — dynamically computed shader. InitDysh, SetDyshShadow, RenderDyshSelf. E.g. water reflections.
SHADOW	P2/shadow.c	Projected shadow. InitShadow, RebuildShadowRegion, InvalidateShadowVifs. Cast via stencil buffer.
EMITTER	P2/emitter.c	Particle emitter. InitEmitter, LoadEmitterFromBrx, CloneEmitter. Smoke, sparks, bubbles.
BLIP	P2/blip.c	Blip — radar/HUD marker. PblipNew, PblipgNew, RemoveBlip. Icons on the map and radar.

Engine systems (BRX / BIS / CLQ / CLOCK / SPLICE)

Type	Source file	Role and key functions
CBinaryInputStream	P2/bis.c	Binary file reader. FOpenFile, FOpenSector, OpenMemory, Close. All Load*FromBrx functions take one as argument.
CBinaryAsyncStream	P2/bas.c	Asynchronous file reader (CD streaming). FOpenFile, StartSpooling. Used for background loading.
BRX	P2/brx.c	Binary Resource — game asset file format. PloNew, LoadSwObjectsFromBrx, LoadOptionFromBrx. Container for everything (mesh, anim, scripts).
CLQ	splice/bif	CLQ — Splice curve. RefOpClqEvaluate, RefOpClqFit. Animation curve callable from Splice scripts.
CLOCK	P2/clock.c	Engine clock. SetClockRate, MarkClockTick, ResetClock, SetClockEnabled. Each system has its own CLOCK for delta time.
ALARM	P2/alarm.c	Timer alarm. InitAlarm, UpdateAlarm, CloneAlarm. Fires callbacks after a delay. SetAlbrkAlarm = alarm breakpoint.
BLOT	P2/screen.c	Blot — 2D sprite rendered on screen. PostBlotsLoad, UpdateBlots, RenderBlots. HUD interface, menus, transition effects.
DMAS	P2/dmas.c	DMA Stream — PS2 DMA manager. StartupDma, AllocGlobal, AllocSw. CPU-free memory transfers.
SLOTHEAP	P2/sloheap.c	Slot-based memory allocator. PvAllocSloheapImpl, CreateSloheapSw. Memory pool for frequent allocations.
PHASE	P2/phasemem.c	Game phase (global state). SetPhase, ClearPhase. Tracks the current game phase (loading, gameplay, cutscene...).
TRANSITION	P2/transition.c	Level/world transition. Set, Execute, ResetWorld. Manages fades and level changes.
THREAD/CRITSECT	P2/thread.c	PS2 thread and critical section. SemaCreate, InitCritSect, EnterCritSect, LeaveCritSect. Multi-thread synchronization.

Splice — embedded Scheme-like scripting engine (P2/splice/)

Splice is an embedded scripting language based on Scheme. Events, cutscenes and AI are scripted in Splice, then evaluated by the C++ engine.

Type	Source file	Role and key functions
<i>CRef</i>	splice/ref.cpp	Reference — a Splice value (int, float, symbol, list, object...). Everything is a CRef. RefOpAdd, RefOpCar, RefOpCdr, CloneTo.
<i>CPair</i>	splice/pair.cpp	Pair (car . cdr) — Scheme list structure. PpairNew, CloneTo, DeletePair. Foundation of all Splice lists.
<i>CFrame</i>	splice/frame.cpp	Environment frame — namespace for Splice variables. AddParent, RefAddBinding, PrefFindBinding.
<i>CSidebag</i>	splice/sidebag.cpp	Binding dictionary. RefAddBinding, FFindBinding. Stores global and local Splice variables.
<i>CGc</i>	splice/gc.cpp	Splice garbage collector. AddRootFrame, PushFrame, Startup, Shutdown. Manages Splice runtime memory.
<i>CSplotheap</i>	splice/splotheap	Splice allocation heap. PvAllocUnsafe, PvAllocClear, PsplotLookup. Dedicated memory pool for the Splice runtime.
<i>RefEval*</i>	splice/eval.cpp	Splice evaluator. RefEvalIf, RefEvalWhile, RefEvalLambda, RefEvalApply. Executes Splice code (if, while, lambda, apply...).
<i>RefOp*</i>	splice/bif.cpp	Built-In Functions. RefOpAdd, RefOpVectorDotProduct, RefOpFindObject, RefOpStartSound. C++ functions callable from Splice.
<i>EOPID</i>	P2/spliceobj.c	Engine Object Property ID — object property accessible from Splice. PeopidFind, RefGetPeopid. Links C++ objects to scripts.
<i>OTYP</i>	P2/spliceobj.c	Object TYPE — Splice object type. RefGetObjectList. Categorizes objects for scripts.

Enemies, gameplay and interactive objects

Type	Source file	Role and key functions
<i>STEPGUARD</i>	P2/stepguard.c	Enemy guard. InitStepguard, CloneStepguard, BindStepguard. Locomotion, detection, patrol. Sub-system of STEP.
<i>SMARTGUARD</i>	P2/smartguard.c	Flashlight guard. InitSmartguard, FFilterSmartguardDetect, UseSmartguardFlashlightTarget. Advanced AI.
<i>UBG</i>	P2/ub.c	Ubiquitous Guard — basic guard. InitUbg, FDetectUbg, OnUbgEnteringSgs. Simplified detection AI.
<i>SBG</i>	P2/sb.c	Super Boss Guard — level boss. UpdateSbgGoal, UpdateSbgSgs, OnSbgEnteringSgs. Boss AI with SGS (State Group System).
<i>MURRAY</i>	P2/murray.c	Murray — Murray character. InitMurray, UpdateMurraySgs. Murray's behavior as an NPC.
<i>SGS</i>	P2/sb.c	State Group System — AI state machine. OnSbgEnteringSgs, SgsNextUbgAI. Each SGS = one AI state (patrol, alert...).
<i>SENSOR</i>	P2/sensor.c	Detection zone. InitSensor, FCheckSensorObject, FIgnoreSensorObject. Triggers events on contact.
<i>CHKPNT</i>	P2/chkpnt.c	Checkpoint — save point. SaveChkmgrCheckpoint, ReturnChkmgrToCheckpoint. Manages Sly's respawn points.
<i>DPRIZE</i>	P2/coin.c	Drop Prize — collectible object (coins, keys). InitDprize, CollectSprize, FFilterDprizeObjects.
<i>SPRIZE</i>	P2/pzo.c	Special Prize — special collectible (master thief bottles). InitSprize, CollectSprize, BindSprize.
<i>TARGET</i>	P2/target.c	Target — attack target. StartupTarget, InitTarget, CloneTarget. Sly aims at enemy TARGETs.
<i>DART</i>	P2/dart.c	Dart — sleep dart. InitDart, CloneDart, LoadDartFromBrx. Projectile fired by Sly with the sleep dart.
<i>ZAP/TZP</i>	P2/zap.c	Zap — electric discharge. InitTzp, FInflctTzpZap, RenderTzpAll. Electrical damage from guards.
<i>WAYPOINT</i>	P2/waypoint.c	AI waypoint. InitWaypoint, SetWaypointRsmg, SetWaypointWps. Guards follow chains of WAYPOINTS.
<i>PATH</i>	P2/path.c	Navigation path. PcgtPointInCbspQuick, ClsgClipEdgeToCbsp. Navigation network for enemies.

AC* suffix guide	B = Bezier C = Component (x/y/z separate) L = Linear W = Weighted BL = Blend (between two types) BLT = BlendTwist BLP = BlendPose Example: ACRBL = Animation Curve Rotation Bezier Linear — blends between Bezier and Linear.
Prototype vs Release	Some prototype (May 2002) modules were removed from the release build: cycle.c (wheeled vehicle), hg.c (hang glider), stepski.c (skiing), rope.c , sc.c , mouthgame.c , menu (debug menu). Their functions may appear in Ghidra but have no .s files in the repo.

17 · decomp.me — tips for getting the most out of it

decomp.me is a collaborative decompilation tool. m2c auto-decompiles the ASM — you refine the output until it matches.

Field / Option	Value / Action	Why
Preset	<i>PS2 > Sly Cooper and the Thievius Raccoonus</i>	Select this exact preset — it configures the correct MIPS GCC compiler and flags to match Sly.

Field / Option	Value / Action	Why
Target assembly	<i>Contents of the .s file</i>	Paste the raw content of the .s (asm/nonmatchings/...). Just the instructions, no file header.
Diff label	<i>Mangled function name</i>	E.g. EvaluateAcpb__FP4ACBPBP3ALOffiP6VECTORT5 — must match the glabel exactly in the .s file
Context	<i>Type definitions used by the function</i>	Paste the needed structs and typedefs from the .h files. Without this m2c cannot type variables.
Context — what	<i>Structs, enums, prototypes of called functions</i>	For EvaluateAcpb: paste struct KVB, struct ACP, struct ACPB, typedef GRFEVAL, prototype EvaluateAkvb
Options > Debug	<i>-g3 (Macro expansions)</i>	Shows source line numbers in generated ASM — helps map ASM lines to C lines
m2c output	<i>Starting point, not the solution</i>	m2c generates valid but ugly C (TEMP_0, TEMP_1...). Rename, retype, restructure until 100%.
Score 100%	<i>Green everywhere in the diff</i>	The CURRENT and TARGET columns must be identical. One red line = not matching.
Fork	<i>Fork button on an existing scratch</i>	If someone already started a function on decomp.me, fork their scratch instead of starting over.
Permanent URL	<i>Share the scratch link</i>	Each scratch has a fixed URL — share on Discord to ask for help on a specific function.

Typical m2c output	How to clean it up
TEMP_0, TEMP_1...	Temporary variables — rename them based on their inferred usage from context
(s32) / (f32) casts	m2c is conservative about types — simplify casts when the struct gives you the type
goto .L...	Jump labels — often convertible to if/else or while when the structure is clear
*(u32 *)(reg + N)	Struct field access — replace with ptr->field once the struct is defined in Context
Shifted registers	If an arg is in the wrong \$a: check float vs integer arg ordering in the signature